

Stopify

Sistema di gestione delle lyrics musicali

Documento dei Requisiti

Corso: Architettura, Reti e Sicurezza
Anno accademico: 2025/2026
Autori: Matteo Barbieri, Ludovico Colussi,
Francesco Martinelli
Università: Università degli Studi del Piemonte Orientale
Corso di laurea: Intelligenza Artificiale e Innovazione Digitale

24 giugno 2026

Indice

1	Descrizione e contesto	2
1.1	Contesto	2
2	Stakeholders	2
3	Architettura hardware e software	2
3.1	Componenti necessari	2
3.2	Infrastruttura	3
3.3	Diagramma a blocchi del sistema	3
4	Architettura di rete	3
4.1	Protocolli e comunicazione	3
4.2	Metodi HTTP utilizzati	4
4.3	Codici di stato restituiti	4
4.4	Schema di errore	4
5	Tecnologie utilizzate	5
5.1	Backend	5
5.2	App Mobile	5
5.3	Protocollo di comunicazione	6
6	Requisiti funzionali e non funzionali	6
6.1	Requisiti funzionali	6
6.2	Requisiti non funzionali	7
7	Use Case Diagram e narrative	8
7.1	Use Case Diagram	8
7.2	Narrative dei casi d'uso	9
8	Glossario e Class Diagram	11
8.1	Class Diagram	11
8.2	Glossario dei termini	12
9	Architettura del sistema	13
9.1	Suddivisione in strati del backend	13
9.2	Suddivisione del frontend mobile	14
9.3	Schema del database	14
10	Autenticazione e Proof of Work	15
10.1	JSON Web Token	15
10.2	Proof of Work	15
11	Analisi degli aspetti di sicurezza	16
11.1	Autenticazione e gestione degli utenti	16
11.2	Protezione dei dati a riposo	16
11.3	Gestione delle vulnerabilità	17
11.4	Crittografia delle comunicazioni	17
11.5	Configurazione del firewall e sicurezza di rete	17
11.6	VPN per gli accessi remoti	17
11.7	Backup e ripristino	18
11.8	Monitoraggio e logging	18
11.9	Conformità legale (GDPR)	18

1 Descrizione e contesto

1.1 Contesto

Un'azienda intende sviluppare un sistema per raccogliere e consultare *testi musicali* (lyrics) associati a brani di qualsiasi artista. L'obiettivo è mantenere un catalogo di testi in due formati:

- **testo piano** (*plain lyrics*): le sole parole della canzone, divise in righe e strofe;
- **testo sincronizzato** (*synced lyrics*, formato LRC): ogni riga è accompagnata dall'istante esatto in cui viene cantata, così che un'app possa evidenziarla a tempo con la musica, come in un karaoke.

Gli utenti possono cercare i testi per titolo, artista e album, e contribuire al catalogo pubblicando di nuovi. Chiunque può pubblicare, anche senza registrarsi: per scoraggiare le pubblicazioni automatiche abusive (*spam*), chi non è registrato deve prima risolvere una piccola sfida crittografica detta *Proof of Work* (sezione 10.2).

Il progetto si chiama **Stopify** ed è composto da due parti che dialogano fra loro:

- un **backend**, cioè il “motore” del sistema: un server che custodisce il database dei testi e risponde alle richieste;
- un **app mobile** per iOS, Android e Web, cioè la parte che l'utente vede e tocca, che mostra i testi e invia le richieste al backend.

2 Stakeholders

Sono considerati tutti i soggetti che hanno un interesse diretto o indiretto nel sistema.

Stakeholder	Descrizione e interesse
Utente anonimo	Visitatore non registrato. Consulta il catalogo, cerca brani per titolo o artista e ne legge il testo (piano o sincronizzato). Può contribuire pubblicando nuovi testi dopo aver superato la sfida Proof of Work.
Utente registrato	Possessore di un account, autenticato tramite JWT. Pubblica testi senza risolvere la Proof of Work; i contributi gli vengono attribuiti (campo <code>submittedBy</code>). Può modificare o cancellare i propri brani.
Amministratore	Utente con privilegi speciali (<code>is_admin</code>). Modera il catalogo: visualizza tutti i brani, inclusi quelli anonimi, li filtra per autore o parola chiave e può modificare o eliminare qualsiasi contenuto.
Sviluppatore	Mantiene il codice del backend e dell'app mobile; ha interesse a una base di codice modulare, testabile e documentata.

Tabella 1: Stakeholders del sistema Stopify.

3 Architettura hardware e software

3.1 Componenti necessari

Per erogare il servizio sono sufficienti due categorie di dispositivi:

- **Lato server**: una macchina (fisica, virtuale o in cloud) con Python 3.10 o superiore. Il prototipo non richiede hardware dedicato: backend e database convivono sullo stesso host, perché SQLite è un database *embedded* che vive in un singolo file (`lyrics.db`) e non necessita di un server separato.

- **Lato client:** lo smartphone (Android o iOS) o il browser dell'utente, su cui gira l'app sviluppata con React Native ed Expo.

Strato	Componente	Ruolo
Server	Python 3.10+	Runtime del backend
Server	Flask 3.1 + Flask-SQLAlchemy 3.1	Framework REST e ORM
Server	Flask-JWT-Extended 4.7	Emissione/verifica dei token JWT
Server	Flask-CORS 6.0	Gestione delle policy CORS
Server	SQLite	Database relazionale embedded
Server	<code>werkzeug.security</code>	Hashing delle password (PBKDF2-SHA256)
Client	React Native 0.81 + Expo SDK 54	Interfaccia mobile cross-platform
Client	expo-router 6	Navigazione fra le schermate
Client	expo-secure-store	Memorizzazione cifrata del token JWT

Tabella 2: Stack software del sistema.

3.2 Infrastruttura

Ambiente di sviluppo. Il backend gira come singolo processo Python sulla macchina del team (porta 5000). L'app mobile è servita dal bundler Metro di Expo e raggiunge il backend tramite l'indirizzo IP della rete locale. Al primo avvio, se il database è vuoto, il sistema lo popola con un piccolo catalogo dimostrativo e crea l'account amministratore predefinito.

3.3 Diagramma a blocchi del sistema

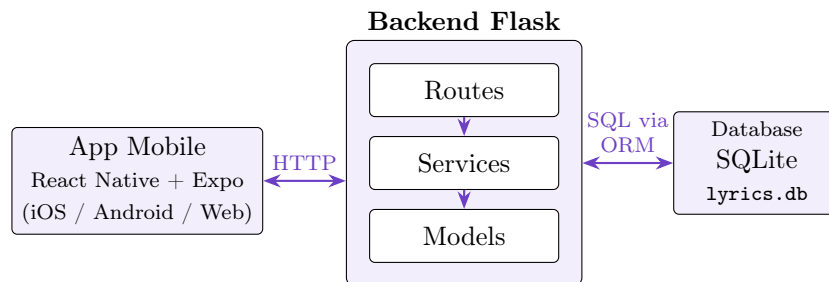


Figura 1: Architettura a blocchi del sistema.

4 Architettura di rete

4.1 Protocolli e comunicazione

Tutte le richieste fra app mobile e backend avvengono via **HTTP/1.1**. Il formato di scambio dei dati è **JSON** con codifica UTF-8, in conformità alla specifica LRCLIB.

I client autenticati allegano il token di accesso nell'header `Authorization: Bearer <jwt>`; le pubblicazioni anonime usano invece l'header dedicato `X-Publish-Token: <prefix>:<nonce>`, che trasporta la soluzione della Proof of Work.

4.2 Metodi HTTP utilizzati

Metodo	Uso	Esempio
GET	Lettura di risorse; non modifica nulla, ripetibile senza effetti collaterali	GET /api/get/3
POST	Creazione di risorse o richiesta di un'azione	POST /api/publish
PUT	Aggiornamento completo di una risorsa esistente	PUT /api/songs/3
DELETE	Rimozione di una risorsa	DELETE /api/songs/3

Tabella 3: Verbi HTTP utilizzati dal sistema.

4.3 Codici di stato restituiti

Codice	Significato	Quando viene restituito
200	OK	Risorsa restituita oppure operazione riuscita
201	Created	Nuova risorsa creata (pubblicazione, registrazione)
400	Bad Request	Body JSON assente o malformato, parametri obbligatori mancanti
401	Unauthorized	Credenziali assenti o non valide; pubblicazione senza JWT né token PoW
403	Forbidden	Proof of Work non superata; modifica/cancellazione di un brano altrui; accesso admin senza privilegi
404	Not Found	Risorsa inesistente (brano o percorso non trovato)
405	Method Not Allowed	Metodo HTTP non previsto per quell'endpoint
409	Conflict	Username o email già in uso in fase di registrazione
422	Unprocessable Entity	Validazione fallita: campi mancanti nella pubblicazione, username/email/password non conformi alle regole
500	Internal Server Error	Errore non gestito lato server

Tabella 4: Codici di stato HTTP restituiti dal sistema.

4.4 Schema di errore

Gli errori gestiti dalle route restituiscono un oggetto JSON con il campo `message`, pensato per essere mostrato direttamente all'utente:

```
{ "message": "Username gia' in uso" }
```

Gli errori globali non intercettati dalle route (404 su percorsi inesistenti, 405, 500) usano invece il campo `error` e non espongono mai dettagli interni del server:

```
{ "error": "Risorsa non trovata" }
```

5 Tecnologie utilizzate

5.1 Backend

Tecnologia	Versione	Motivazione della scelta
Python	3.10+	Linguaggio ad alto livello con un ecosistema maturo per lo sviluppo web e la gestione dei dati.
Flask	3.1	Framework REST leggero e flessibile; non impone strutture rigide e permette di organizzare il codice secondo il pattern Routes → Services → Models.
Flask-SQLAlchemy	3.1	ORM che astrae il database in classi Python; garantisce query parametrizzate (prevenzione delle SQL injection) e facilita la migrazione futura verso altri DBMS.
SQLite	embedded	Database relazionale senza server separato, adatto a sviluppo e prototipazione.
Flask-JWT-Extended	4.7	Gestione nativa degli access token JWT con algoritmo HS256, verifica automatica di firma e scadenza.
Flask-CORS	6.0	Abilita le intestazioni CORS necessarie alle richieste cross-origin dell'app (e del browser in sviluppo).
<code>werkzeug.security</code>	—	Hashing delle password con PBKDF2-SHA256 e salt casuale; incluso nell'ecosistema Flask, senza dipendenze aggiuntive.

Tabella 5: Stack tecnologico del backend.

5.2 App Mobile

Tecnologia	Versione	Motivazione della scelta
React Native	0.81	Genera interfacce native per iOS e Android da un'unica codebase JavaScript/TypeScript, riducendo i costi rispetto a due app separate.
Expo SDK	54	Toolchain che semplifica build, ricarica istantanea e accesso alle funzioni del telefono (storage sicuro, ecc.) senza configurare Xcode o Android Studio.
expo-router	6	Navigazione <i>file-based</i> : ogni file nella cartella <code>app/</code> diventa automaticamente una schermata raggiungibile.
TypeScript	—	Tipizzazione statica che riduce i bug a runtime; utile per definire i tipi delle risposte REST.
expo-secure-store	—	Salva il token JWT nel Keychain (iOS) o Keystore (Android), le aree di memoria cifrate del sistema operativo; sul Web ricade su <code>localStorage</code> .

Tabella 6: Stack tecnologico dell'app mobile.

5.3 Protocollo di comunicazione

Il trasferimento dati avviene tramite HTTP/REST/JSON come descritto nella sezione 4. In ambiente di sviluppo le comunicazioni avvengono in HTTP non cifrato sulla rete locale. (sezione 11.4).

6 Requisiti funzionali e non funzionali

6.1 Requisiti funzionali

I requisiti contrassegnati con (*LRCLIB*) corrispondono agli endpoint richiesti dalla traccia; gli altri sono estensioni sviluppate per completare l'esperienza d'uso.

ID	Nome	Descrizione
FR1	Ricerca per signature (<i>LRCLIB</i>)	Il sistema deve consentire di cercare un testo tramite la sua <i>signature</i> : <code>track_name + artist_name</code> , con filtro opzionale su <code>album_name</code> e <code>duration</code> (tolleranza ± 2 secondi). Realizzato da <code>GET /api/get</code> .
FR2	Ricerca per signature, versione cached (<i>LRCLIB</i>)	Come FR1, ma limitata alla cache locale. Realizzato da <code>GET /api/get-cached</code> ; nella nostra implementazione coincide con FR1 perché il database locale è la cache (non esiste fallback verso sorgenti esterne).
FR3	Recupero per ID (<i>LRCLIB</i>)	Deve essere possibile recuperare un testo conoscendo il suo identificativo univoco. Realizzato da <code>GET /api/get/<id></code> .
FR4	Ricerca generica (<i>LRCLIB</i>)	L'utente deve poter cercare testi per parola chiave (<code>q</code> , confrontata con titolo e artista) o per combinazione di titolo, artista e album; massimo 50 risultati. Realizzato da <code>GET /api/search</code> .
FR5	Richiesta di challenge (<i>LRCLIB</i>)	Il sistema deve emettere su richiesta una sfida Proof of Work { <code>prefix</code> , <code>target</code> }. Realizzato da <code>POST /api/request-challenge</code> .
FR6	Pubblicazione anonima (<i>LRCLIB</i>)	Un utente non registrato deve poter pubblicare un nuovo testo dopo aver risolto la sfida. Realizzato da <code>POST /api/publish</code> con header <code>X-Publish-Token</code> .
FR7	Pubblicazione autenticata	Un utente autenticato deve poter pubblicare senza Proof of Work; il brano viene attribuito al suo username nel campo <code>submittedBy</code> . Realizzato da <code>POST /api/publish</code> con header <code>Authorization</code> .
FR8	Registrazione utente	Deve essere possibile creare un account con username, email e password (validati lato server). Realizzato da <code>POST /auth/register</code> .
FR9	Login utente	L'utente deve potersi autenticare con username o email più password, ricevendo un access token JWT. Realizzato da <code>POST /auth/login</code> .
FR10	Dati dell'utente corrente	L'utente autenticato deve poter recuperare i propri dati di profilo. Realizzato da <code>GET /auth/me</code> .
FR11	Elenco dei propri brani	L'utente autenticato deve poter ottenere la lista dei testi da lui pubblicati. Realizzato da <code>GET /api/me/songs</code> .

ID	Nome	Descrizione
FR12	Modifica di un proprio brano	L'utente autenticato deve poter modificare un brano di cui è autore. Realizzato da <code>PUT /api/songs/<id></code> .
FR13	Cancellazione di un proprio brano	L'utente autenticato deve poter cancellare un brano di cui è autore. Realizzato da <code>DELETE /api/songs/<id></code> .
FR14	Pannello amministratore	Un utente con flag <code>is_admin</code> deve poter consultare l'intero catalogo, inclusi i brani anonimi, con filtri per parola chiave (<code>q</code>), per soli brani anonimi (<code>anonymous</code>) e per autore (<code>user_id</code>); deve inoltre poter modificare o eliminare qualsiasi brano. Realizzato da <code>GET /admin/songs</code> e dagli endpoint di FR12/FR13, che per gli admin non applicano il vincolo di proprietà.
FR15	Esplorazione del catalogo	Il sistema deve fornire un elenco paginato dei brani, ordinabile per data di inserimento, titolo o artista. Realizzato da <code>GET /api/explore</code> .
FR16	Lettura del testo sincronizzato	L'app mobile deve mostrare il testo sincronizzato a tempo, evidenziando la riga corrente durante lo scorrimento del brano.
FR17	Health check	Deve essere disponibile un endpoint per verificare lo stato del backend, mostrato nella home dell'app. Realizzato da <code>GET /api/health</code> .

Tabella 7: Requisiti funzionali.

6.2 Requisiti non funzionali

ID	Tipo	Descrizione	Riferito a
NFR1	Sicurezza	Le password devono essere memorizzate esclusivamente come hash PBKDF2-SHA256 con salt casuale; mai in chiaro.	Backend, DB
NFR2	Sicurezza	Gli endpoint protetti devono richiedere un JWT valido, firmato (HS256) e non scaduto (durata 7 giorni).	Backend
NFR3	Sicurezza	Le pubblicazioni anonime devono essere accettate solo previa verifica della Proof of Work; la verifica è <i>stateless</i> (nessuna challenge persistita sul database).	Backend
NFR4	Sicurezza	Il login fallito deve restituire un messaggio unico ("Credenziali non valide"), senza rivelare se esiste l'username, per prevenire la <i>user enumeration</i> .	Backend
NFR5	Privacy	Il sistema non deve associare dati personali alle pubblicazioni anonime effettuate via PoW.	Backend, DB

ID	Tipo	Descrizione	Riferito a
NFR6	Compatibilità	Lo scambio dati deve avvenire esclusivamente in JSON UTF-8 su HTTP/REST, secondo lo schema LRCLIB.	Sistema
NFR7	Portabilità	L'app client deve funzionare su Android, iOS e Web da un'unica codebase; il backend su qualsiasi sistema con Python ≥ 3.10 .	Sistema
NFR8	Manutenibilità	Il backend deve essere organizzato in strati separati (routes, services, models, utils) per testabilità e leggibilità.	Backend

Tabella 8: Requisiti non funzionali.

7 Use Case Diagram e narrative

7.1 Use Case Diagram

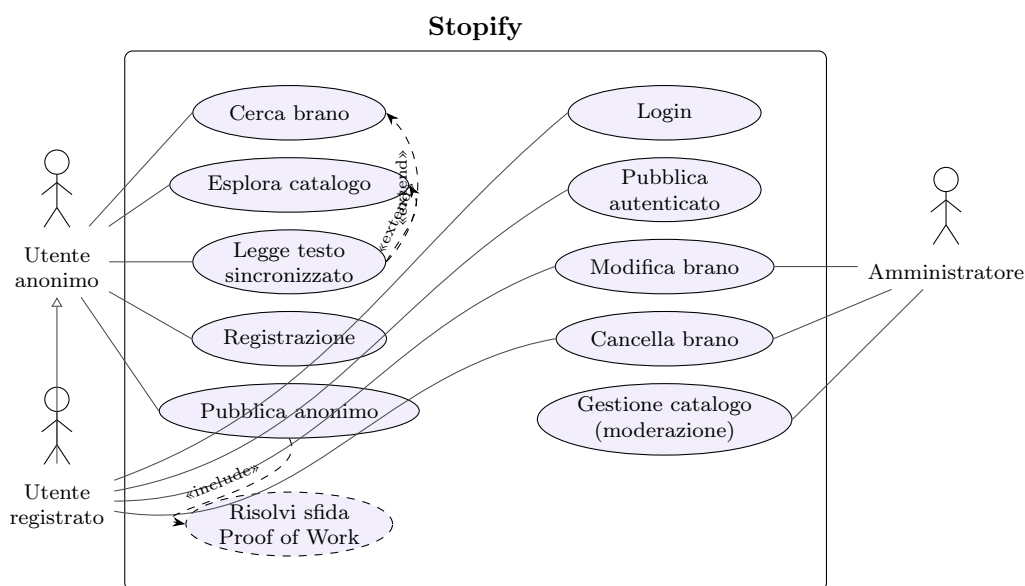


Figura 2: Use Case Diagram di Stopify. L'utente registrato eredita le funzioni dell'utente anonimo ma pubblica direttamente senza completare la sfida Proof of Work; l'amministratore modera il catalogo.

7.2 Narrative dei casi d'uso

UC1 — Ricerca di un brano

Scope: catalogo lyrics.

Level: user-goal.

Intention in context: l'utente vuole leggere il testo di un brano che gli è venuto in mente.

Primary actor: Utente anonimo.

Support actor: Backend Flask.

Stakeholders' interest:

- *Utente*: trovare velocemente il testo di un brano.
- *Sistema*: servire risultati rilevanti con bassa latenza.

Precondition: il backend è raggiungibile.

Minimum guarantees: nessuna modifica dello stato del sistema.

Success guarantees: l'utente vede il testo del brano richiesto.

Trigger: l'utente apre la schermata "Cerca".

Main success scenario:

1. L'utente digita una stringa nel campo di ricerca.
2. L'app invia GET `/api/search?q=<stringa>`.
3. Il backend restituisce l'elenco dei brani il cui titolo o artista contiene la stringa.
4. L'utente seleziona un brano.
5. L'app invia GET `/api/get/<id>` e mostra il testo.

Extensions:

- *3a*: nessun risultato $\Rightarrow$ l'app mostra lo stato vuoto "Nessun risultato".

UC2 — Registrazione di un nuovo account

Scope: autenticazione.

Level: user-goal.

Intention in context: l'utente vuole creare un account per contribuire senza dover risolvere la Proof of Work a ogni pubblicazione e per vedersi attribuiti i contributi.

Primary actor: Utente anonimo.

Precondition: l'utente non è autenticato.

Success guarantees: è stato creato un nuovo record **User** e l'utente è autenticato.

Main success scenario:

1. L'utente apre la schermata "Crea un account".
2. Inserisce username, email e password.
3. L'app valida i campi lato client (formato e lunghezza minima).
4. L'app invia POST `/auth/register` con il payload.
5. Il backend valida nuovamente, calcola l'hash della password (PBKDF2-SHA256), crea il record e restituisce l'access token JWT.
6. L'app salva il token nello storage sicuro e mostra la home da autenticato.

Extensions:

- *5a*: username o email già presenti $\Rightarrow$ il backend risponde 409 e l'app mostra il messaggio di errore.
- *5b*: username, email o password non conformi alle regole $\Rightarrow$ 422 con il dettaglio della violazione.

UC3 — Pubblicazione anonima con Proof of Work

Scope: contenuti, sicurezza.

Level: user-goal.

Intention in context: l'utente vuole condividere il testo di un brano senza creare un account.

Primary actor: Utente anonimo.

Precondition: l'utente conosce titolo, artista e testo del brano.

Minimum guarantees: nessun brano viene pubblicato se la Proof of Work non è superata.

Success guarantees: un nuovo record `Song` è presente nel database con `user_id = NULL`.

Main success scenario:

1. L'utente apre la schermata "Pubblica" e compila il form.
2. L'app invia `POST /api/request-challenge` e riceve `{prefix, target}`.
3. L'app risolve la sfida cercando, per tentativi, un `nonce` tale che $\text{SHA-256}(\text{prefix} + \text{nonce}) \leq \text{target}$.
4. L'app invia `POST /api/publish` con il body del brano e l'header `X-Publish-Token: prefix:nonce`.
5. Il backend ricalcola l'hash, verifica la disuguaglianza (verifica *stateless*, sezione 10.2), crea il brano e risponde 201 con il record creato.

Extensions:

- *5a*: la verifica fallisce (nonce errato) $\Rightarrow$ 403 Forbidden.
- *5b*: campi obbligatori mancanti (titolo, artista o testo) $\Rightarrow$ 422 con il dettaglio.

UC4 — Pubblicazione autenticata

Scope: contenuti.

Level: user-goal.

Primary actor: Utente registrato.

Precondition: l'utente è autenticato (access token valido).

Success guarantees: il nuovo brano viene creato con `user_id` valorizzato e `submittedBy` pari allo username dell'autore.

Main success scenario:

1. L'utente apre la schermata "Pubblica" e compila il form.
2. L'app, rilevando il contesto autenticato, salta la richiesta di challenge.
3. L'app invia `POST /api/publish` con header `Authorization: Bearer <jwt>`.
4. Il backend identifica l'utente dal token e crea il brano attribuendoglielo.

Extensions:

- *4a*: token scaduto o non valido $\Rightarrow$ 401; l'app svuota la sessione locale e reindirizza al login.

UC5 — Modifica di un brano pubblicato

Scope: contenuti, autorizzazione.

Level: user-goal.

Primary actor: Utente registrato.

Precondition: l'utente è autenticato ed è l'autore del brano.

Minimum guarantees: un brano non può essere modificato da chi non ne è l'autore (salvo amministratori).

Main success scenario:

1. L'utente apre "I miei testi".
2. L'app chiama `GET /api/me/songs` e mostra l'elenco.
3. L'utente tocca l'icona di modifica su un brano.
4. L'app naviga al form precompilato.
5. Dopo le modifiche, invia `PUT /api/songs/<id>`.
6. Il backend verifica che il brano appartenga all'utente, aggiorna i campi e risponde 200.

Extensions:

- *6a*: il brano appartiene a un altro utente $\Rightarrow$ 403 Forbidden.
- *6b*: il brano è anonimo (`user_id = NULL`) $\Rightarrow$ 403 (non modificabile dagli utenti normali).

UC6 — Cancellazione di un brano

Scope: contenuti, autorizzazione.

Primary actor: Utente registrato.

Success guarantees: il record **Song** viene rimosso dal database e non è più reperibile via API.

Main success scenario:

1. L'utente apre "I miei testi".
2. Tocca l'icona di cancellazione su un brano.
3. L'app mostra un alert di conferma.
4. Su conferma, invia `DELETE /api/songs/<id>`.
5. Il backend verifica la proprietà, cancella il record e risponde 200.

Extensions:

- *4a*: l'utente annulla l'alert $\Rightarrow$ nessuna azione.
- *5a*: brano non di proprietà $\Rightarrow$ 403.

UC7 — Gestione del catalogo (Amministratore)

Scope: moderazione contenuti.

Level: user-goal.

Intention in context: l'amministratore vuole visionare e moderare l'intero catalogo, inclusi i brani anonimi che un utente normale non può gestire.

Primary actor: Amministratore.

Precondition: l'amministratore è autenticato con un account avente `is_admin = true`.

Minimum guarantees: un utente non amministratore non può accedere alle funzioni di moderazione.

Success guarantees: l'amministratore visualizza il catalogo filtrato e può modificare o eliminare qualsiasi brano.

Main success scenario:

1. L'amministratore apre la schermata "Admin".
2. L'app invia `GET /admin/songs` con il JWT nell'header.
3. Il backend verifica il flag `is_admin` dell'utente.
4. Il backend restituisce l'elenco completo dei brani, inclusi quelli anonimi.
5. L'amministratore può filtrare per testo (`q`), per soli brani anonimi (`anonymous=true`) o per autore (`user_id`).
6. Per eliminare un brano, l'app invia `DELETE /api/songs/<id>`: il backend accetta perché rileva il flag `is_admin`, anche se il brano non appartiene all'amministratore.

Extensions:

- *3a*: JWT mancante o scaduto $\Rightarrow$ 401.
- *3b*: utente autenticato ma non admin $\Rightarrow$ 403.

8 Glossario e Class Diagram

8.1 Class Diagram

Il modello di dominio è volutamente essenziale: due sole entità, **User** e **Song**, legate da un'associazione opzionale. Artista e album sono memorizzati come semplici attributi testuali del brano (modello "piatto", coerente con la specifica LRCLIB): il sistema non prevede pagine artista o album né metadati dedicati, quindi tabelle separate non porterebbero alcun vantaggio funzionale.

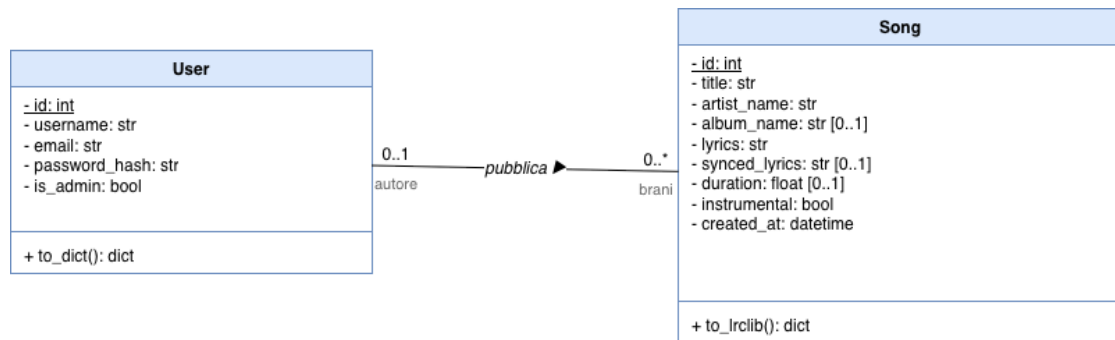


Figura 3: Class Diagram (UML) del dominio di Stopify. Un utente può aver pubblicato zero o più brani; un brano può appartenere a un utente oppure essere anonimo (`user_id = NULL`).

8.2 Glossario dei termini

Termine	Descrizione
Lyric / Testo	Insieme delle parole cantate in un brano musicale. Memorizzato in forma piana (<code>plainLyrics</code>) e/o sincronizzata (<code>syncedLyrics</code>).
Plain lyrics	Testo in chiaro privo di marcatori temporali, suddiviso in righe e strofe.
Synced lyrics (LRC)	Testo con timestamp del tipo <code>[mm:ss.xx]</code> che indicano il momento esatto in cui ogni riga viene cantata. Internamente è memorizzato come JSON e convertito nel formato LRC standard al momento della risposta.
LRCLIB	Servizio pubblico di lyrics il cui schema di API è emulato dal backend di Stopify, come richiesto dalla traccia.
Signature di un brano	Combinazione (titolo, artista, eventualmente album e durata) usata per identificare univocamente un brano nelle ricerche.
Song / Brano	Entità che rappresenta un singolo brano: titolo, nome dell'artista, eventuale album, testo piano e sincronizzato, durata, flag strumentale ed eventuale utente autore.
User / Utente	Utente registrato; possiede credenziali (username, email, hash della password) e l'eventuale flag <code>is_admin</code> . Può pubblicare brani a lui attribuiti.
JWT	JSON Web Token: "tessera digitale" firmata dal server che contiene l'identità dell'utente e una scadenza. Il client la allega all'header <code>Authorization</code> per autenticare le richieste.
Access token	JWT con durata di 7 giorni usato per autenticare le chiamate API. Alla scadenza l'utente deve effettuare nuovamente il login.
Proof of Work (PoW)	Sfida crittografica che il client risolve per tentativi prima di poter inviare una pubblicazione anonima. Impone un costo computazionale che disincentiva lo spam massivo.
Nonce	Numero che il client fa variare durante la ricerca della soluzione della PoW, finché l'hash non soddisfa la condizione richiesta.

Termine	Descrizione
Hash	“Impronta digitale” di un dato: una funzione matematica unidirezionale che trasforma qualsiasi input in una stringa di lunghezza fissa, dalla quale è praticamente impossibile risalire all’input originale.
submittedBy	Campo della risposta che indica lo username dell’autore di un brano (<code>null</code> se anonimo).
ORM	Object-Relational Mapping: mappatura automatica fra classi Python e tabelle del database, realizzata tramite SQLAlchemy.
REST	Stile architetturale per le API web: le risorse sono identificate da URL e manipolate con i metodi standard HTTP (GET, POST, PUT, DELETE).
CORS	Cross-Origin Resource Sharing: meccanismo del browser che regola le richieste provenienti da un dominio diverso da quello del backend.

Tabella 9: Glossario dei termini.

9 Architettura del sistema

9.1 Suddivisione in strati del backend

Il backend è organizzato secondo il pattern **Routes** → **Services** → **Models**: ogni strato ha una responsabilità precisa e parla solo con lo strato adiacente. Questo rende la logica di business testabile indipendentemente dal livello HTTP.

Strato	Responsabilità
app/routes/	Punti di ingresso HTTP. Lettura dei parametri e del payload, dispatch ai servizi, formattazione della risposta JSON, mapping degli errori nei codici di stato. Tre blueprint: <code>/api/*</code> (LRCLIB + estensioni), <code>/auth/*</code> , <code>/admin/*</code> .
app/services/	Logica di business: ricerca e pubblicazione dei brani, registrazione e login con relative validazioni, generazione e verifica della Proof of Work, controlli di autorizzazione e transazioni sul database.
app/models/	Classi SQLAlchemy <code>User</code> e <code>Song</code> , con i metodi di serializzazione verso il formato LRCLIB (<code>to_lrclib</code> , <code>to_dict</code>).
app/utils/	Helper trasversali: encoder/decoder fra formato LRC e JSON.
app/config.py	Configurazione <i>env-aware</i> : URI del database, chiave JWT, scadenza dei token, limite dimensionale dei payload, difficoltà della PoW, credenziali admin di default.
app/extensions.py	Istanze singleton delle estensioni Flask (<code>db</code> , <code>jwt</code> , <code>cors</code>).
app/errors.py	<code>AppError</code> , l’eccezione di dominio unificata convertita automaticamente in risposta JSON di errore.
main.py	Entrypoint: creazione dell’app, seeding del catalogo iniziale e dell’utente admin, avvio del server di sviluppo.

Tabella 10: Layout del backend.

9.2 Suddivisione del frontend mobile

Cartella / File	Ruolo
app/	Schermate (routing file-based di expo-router).
app/_layout.tsx	Layout radice; registra l'AuthProvider.
app/index.tsx	Home con stato di connessione e accesso al profilo.
app/search.tsx	Ricerca interattiva nel catalogo.
app/explore.tsx	Catalogo paginato con ordinamento.
app/song/[id].tsx	Dettaglio del brano con testo piano e sincronizzato.
app/publish.tsx	Form di pubblicazione (JWT o PoW), con controllo anti-duplicato.
app/login.tsx, register.tsx	Schermate di autenticazione.
app/my-lyrics.tsx	Elenco delle pubblicazioni dell'utente.
app/edit-song/[id].tsx	Form di modifica di un proprio brano.
app/admin.tsx	Pannello amministratore (solo <code>is_admin</code>).
src/api.ts	Client REST per tutte le chiamate al backend, incluso il solver della PoW.
src/AuthContext.tsx	Stato globale dell'autenticazione (React Context).
src/storage.ts	Persistenza sicura del token, cross-platform.
src/sha256.ts	Implementazione client di SHA-256 per il calcolo del nonce.

Tabella 11: Layout del frontend mobile.

9.3 Schema del database

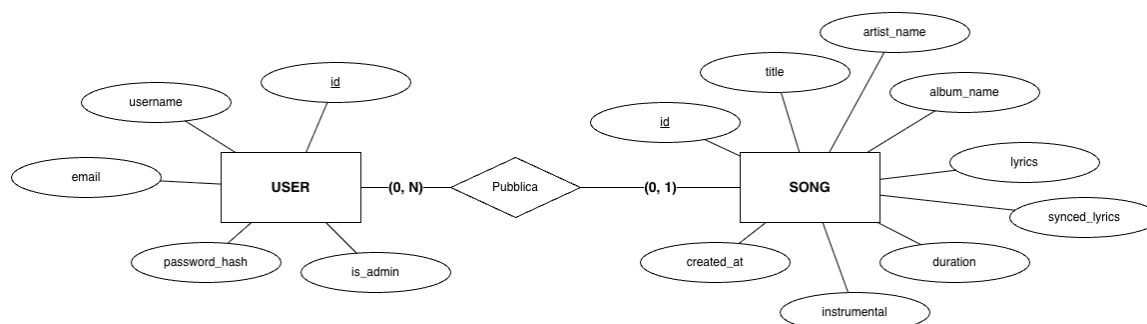


Figura 4: Schema entità-relazione del database (notazione di Chen).

Il database è composto da due tabelle:

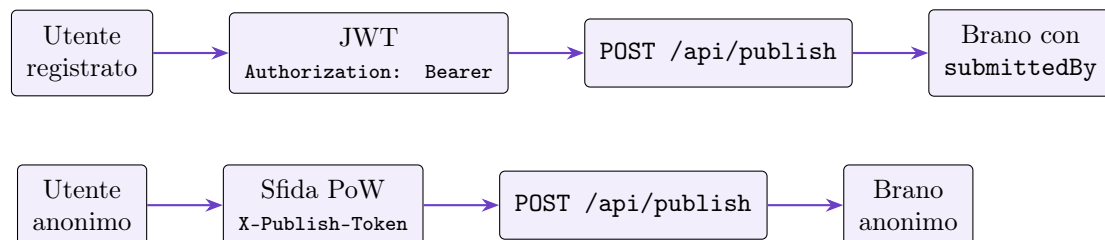
- **users** (`id`, `username`, `email`, `password_hash`, `is_admin`) — utenti registrati. Username ed email sono univoci (il controllo è case-insensitive); `is_admin` (booleano, default falso) abilita le funzioni di moderazione. La password è memorizzata esclusivamente come hash.
- **songs** (`id`, `title`, `artist_name`, `album_name`, `lyrics`, `synced_lyrics`, `duration`, `instrumental`, `user_id`, `created_at`) — brani del catalogo, con testo piano e sincronizzato. `user_id` è una foreign key *nullable* verso **users**: quando vale NULL il brano è anonimo (pubblicato tramite Proof of Work).

Scelte progettuali rilevanti:

- *Modello piatto*: `artist_name` e `album_name` sono colonne della tabella `songs` anziché tabelle separate, in coerenza con la specifica LRCLIB e con i casi d'uso effettivi (nessuna pagina artista/album).
- *Proof of Work stateless*: le sfide PoW **non** vengono salvate nel database; la verifica è puramente computazionale (sezione 10.2). Questo mantiene lo schema minimale ed evita tabelle di stato da ripulire.
- *Testo sincronizzato*: `synced_lyrics` è memorizzato come stringa JSON e convertito da/-verso il formato LRC standard (`[mm:ss.xx] testo`) da un modulo dedicato, disaccoppiando il formato di archiviazione da quello di trasporto.

10 Autenticazione e Proof of Work

La pubblicazione di un brano segue un **modello ibrido**, che concilia l'apertura ai contributi anonimi prevista da LRCLIB con la possibilità di avere contributi attribuiti:



10.1 JSON Web Token

Parametro	Valore
Algoritmo di firma	HS256 (HMAC-SHA256, chiave simmetrica)
Durata dell'access token	7 giorni
Identità trasportata	<code>user.id</code>
Trasporto	header <code>Authorization: Bearer <token></code>

Tabella 12: Parametri JWT.

L'app mobile conserva il token con `expo-secure-store` (Keychain su iOS, Keystore su Android). Alla ricezione di un 401 il client svuota la sessione locale e reindirizza al login.

10.2 Proof of Work

La Proof of Work è il meccanismo previsto dalla specifica LRCLIB per *rallentare lo spam* di pubblicazioni anonime senza richiedere registrazione né CAPTCHA visivi.

Algoritmo.

1. Il client chiama `POST /api/request-challenge` e riceve `{prefix, target}`: `prefix` è una stringa casuale, `target` è la soglia da rispettare.
2. Il client cerca un numero intero `nonce` tale che, confrontando le stringhe esadecimali,

$$\text{SHA-256}(\text{prefix} + \text{nonce}) \leq \text{target}.$$

La ricerca avviene per tentativi incrementali (`nonce = 0, 1, 2, \dots`): non esiste scorciatoia, perché l'hash non è prevedibile.

3. Il client invia `prefix:nonce` nell'header `X-Publish-Token` della POST `/api/publish`.
4. Il server ricalcola l'hash e verifica la disuguaglianza: un solo calcolo, istantaneo. La verifica è **stateless**: nessuna challenge viene salvata sul database.

Difficoltà. Con `POW_DIFFICULTY = 4` il target richiede 4 zeri esadecimali iniziali: in media servono circa $16^4 \approx 65\,000$ tentativi, ossia $\sim 0,5$ secondi su uno smartphone moderno. Il valore è configurabile in `app/config.py`: ogni zero in più moltiplica per 16 il lavoro richiesto al client, senza alcun costo aggiuntivo per il server.

Asimmetria del costo. Il punto di forza del meccanismo è l'asimmetria: il client deve fare decine di migliaia di calcoli per trovare la soluzione, il server uno solo per verificarla. La scelta **stateless** mantiene il backend semplice e privo di stato da gestire; il rovescio della medaglia (la possibilità di riutilizzare una soluzione già trovata) è discusso fra le vulnerabilità nella sezione ??.

11 Analisi degli aspetti di sicurezza

L'analisi copre i tre pilastri classici della sicurezza informatica — *confidenzialità* (i dati sono leggibili solo da chi ne ha diritto), *integrità* (i dati non possono essere alterati di nascosto) e *disponibilità* (il servizio resta accessibile) — ed è suddivisa fra **controlli già implementati nel codice** e **controlli previsti come future work**, rinviati alla fase di messa in produzione. Le variabili critiche del sistema sono: le credenziali degli utenti, la chiave di firma dei JWT (`JWT_SECRET_KEY`) e il file del database.

11.1 Autenticazione e gestione degli utenti

Implementato:

- Password con hashing **PBKDF2-SHA256** e salt casuale per ogni utente (default di `werkzeug.security`). Il salt fa sì che due utenti con la stessa password abbiano hash diversi, vanificando gli attacchi con tabelle precalcolate.
- Validazione lato server: `username ^ [A-Za-z0-9_]{3,30}$`, email in formato standard normalizzata in minuscolo, password di almeno 8 caratteri con almeno una cifra.
- Univocità di username ed email verificata in modo case-insensitive.
- Risposta unificata al login fallito ("Credenziali non valide"), sia che l'utente non esista sia che la password sia errata: impedisce a un attaccante di scoprire quali username sono registrati (*user enumeration*).
- L'account amministratore iniziale è creato al primo avvio con credenziali configurabili via variabili d'ambiente (`ADMIN_USERNAME`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`); i valori di default vanno sostituiti in produzione.

11.2 Protezione dei dati a riposo

Implementato:

- Le password esistono solo come hash; il valore in chiaro non viene mai salvato, loggato né restituito dalle API.
- I JWT non vengono persistiti lato server: sono *stateless* e autocontenuti.
- Sul dispositivo dell'utente il token è salvato nelle aree cifrate del sistema operativo (Keychain/Keystore).

Future work:

- Il database SQLite vive in un singolo file (`lyrics.db`): in produzione va protetto con permessi di filesystem riservati all'utente del processo Flask, escluso dal version control e, se necessario, cifrato (cifatura del filesystem o SQLCipher).

11.3 Gestione delle vulnerabilità

Vengono qui raccolte le principali vulnerabilità note del prototipo, con il relativo rischio e la mitigazione prevista. Trattandosi di un sistema sviluppato a scopo didattico, alcune contromisure sono rimandate alla (eventuale) fase di messa in produzione (*future work*).

- **Replay della soluzione Proof of Work.** La verifica della PoW è *stateless*: nessuna challenge viene salvata sul database. Di conseguenza una coppia `prefix:nonce` valida non viene invalidata dopo l'uso e può essere riutilizzata per inviare più pubblicazioni anonime, riducendo l'efficacia anti-spam del meccanismo.
Mitigazione (future work): associare a ogni challenge una scadenza breve e tenere traccia dei nonce già consumati (per esempio in una cache in memoria con TTL), così che ciascuna soluzione sia utilizzabile una sola volta.
- **Assenza di rate limiting.** Gli endpoint di autenticazione (`/auth/login`, `/auth/register`) e di pubblicazione non impongono limiti sul numero di richieste per client. Questo espone il sistema ad attacchi di forza bruta sulle password e a invii massivi automatizzati.
Mitigazione (future work): introdurre un rate limiter per indirizzo IP (per esempio `Flask-Limiter`) e un backoff progressivo sui tentativi di login falliti.
- **Segreti e credenziali di default.** In assenza di configurazione, il sistema usa una chiave di firma JWT prevedibile (`JWT_SECRET_KEY = "super-secret-key"`) e un account amministratore con password nota (`ADMIN_PASSWORD = "admin123"`). Un attaccante che conosca questi default potrebbe forgiare token validi o accedere come admin.
Mitigazione: in produzione i valori vanno obbligatoriamente sovrascritti tramite variabili d'ambiente (`JWT_SECRET_KEY`, `ADMIN_PASSWORD`), generando segreti casuali e robusti.
- **Politica CORS permissiva.** Le intestazioni CORS sono abilitate senza restrizione sulle origini, utile in sviluppo ma rischioso in produzione perché consente a qualsiasi sito di invocare le API dal browser dell'utente.
Mitigazione (future work): limitare le origini consentite alla sola applicazione client ufficiale.
- **Trasporto in chiaro (HTTP).** In sviluppo le comunicazioni non sono cifrate (sezione ??), esponendo token e credenziali a intercettazione su reti non fidate.
Mitigazione (future work): servire l'applicazione esclusivamente via HTTPS dietro un reverse proxy con certificato TLS.

11.4 Crittografia delle comunicazioni

Stato attuale: in sviluppo le comunicazioni avvengono su HTTP non cifrato, accettabile perché il traffico non lascia la rete locale fidata.

11.5 Configurazione del firewall e sicurezza di rete

Stato attuale: nessun firewall specifico; il server è in ascolto su `0.0.0.0:5000` ed è raggiungibile solo dalla rete locale.

Future work:

- Firewall host-based (per esempio `ufw` su Linux) che esponga solo le porte 80 e 443; la porta 5000 di Flask resta chiusa verso l'esterno e raggiungibile solo dal reverse proxy in loopback:

11.6 VPN per gli accessi remoti

- Negli scenari in cui il backend non debba essere esposto pubblicamente (deployment interno aziendale), il server può essere reso raggiungibile solo tramite VPN (WireGuard o

OpenVPN): un “tunnel” cifrato che fa apparire i dispositivi autorizzati come se fossero sulla stessa rete locale.

- Il pannello di amministrazione (`/admin/*`) è il candidato naturale per questa restrizione: il reverse proxy può essere configurato per accettare richieste sul prefisso `/admin` solo da indirizzi IP della VPN.
- In un team distribuito, la VPN garantisce che le chiamate di test non transitino in chiaro su reti pubbliche.

11.7 Backup e ripristino

- In sviluppo il database è un singolo file: il backup consiste nella copia di `lyrics.db`.
- In produzione si raccomanda un backup notturno automatizzato.
- Il codice sorgente è versionato con Git: il ripristino completo dell'applicazione richiede solo il repository e l'ultimo backup del database.

11.8 Monitoraggio e logging

Stato attuale: in sviluppo Flask stampa su stdout le richieste HTTP con codice di stato e indirizzo sorgente.

11.9 Conformità legale (GDPR)

- I dati personali raccolti sono minimi: username, indirizzo email e hash della password. Non sono trattate categorie speciali di dati (art. 9 GDPR).
- Gli utenti possono consultare i propri dati tramite `GET /auth/me` (diritto di accesso, art. 15).
- **Diritto all'oblio** (art. 17): non è ancora esposto un endpoint di cancellazione dell'account. Come future work: `DELETE /auth/me`, che rimuova il record `User` e anonimizzi i brani associati impostando `user_id = NULL` (i testi, non essendo dati personali, possono restare nel catalogo).
- I log di produzione non devono contenere dati personali in chiaro: è sufficiente registrare l'ID numerico dell'utente anziché username o email.
- La privacy policy deve essere mostrata all'utente al momento della registrazione, prima della raccolta dei dati.
- Le pubblicazioni anonime non sono associate ad alcun dato personale, in linea con il principio di minimizzazione (art. 5).